

Report: multiple issue for the Tomasulo simulator

TL;DR

We started from the course's single-issue Python simulator ([tomasulo-python.zip](#)). On small inputs it computed a wrong register value, wrote memory too early, lost a store, ran a multiplication one cycle short and crashed; we fixed those five bugs first (Appendix A). Then we added what the assignment asks for, issuing 1 to 4 instructions per cycle in program order, plus three parts of the machine the handout describes but the code did not have:

- the branch unit: a 1-bit predictor in an 8-entry BTB, speculative execution, and recovery from mispredictions;
- the memory: 256 bytes, with **Ld** and **Sd** moving single-precision values;
- the load rule: a load does not wait for older stores whose address is still unknown. We added the check that makes this safe: a load that read a value too early is executed again.

We ran our 25 test inputs at issue widths 1–4, each in three CDB settings: 300 runs (Appendix C). In all 288 runs of the 24 programs that finish, the simulator ends with the same registers and memory as a simple one-instruction-at-a-time interpreter, and every committed instruction keeps to the limits and orderings of the timing rules. The other 12 runs are the handout's sample, which never ends; there the instructions committed in the first 200 cycles are compared. For 7 small tests we computed 19 instruction tables by hand, and they match exactly. 2000 random programs pass the same checks at all four widths. We also put 25 deliberate mistakes into copies of the code, and the checks catch every one.

How much a wider issue helps depends on the program and on the number of CDBs (section 1.4). The handout lists one CDB. The sample report lets as many results write back and commit per cycle as the issue width. That is our default, and section 3 gives every test in both settings. With the default setting, eight independent instructions take 11 cycles at width 1 and 5 at width 4, and a DAXPY loop (28 instructions) goes from 41 to 31 cycles. With one CDB, the eight instructions take 11 cycles at every width and DAXPY goes from 41 to 33.

Contents

- [1. Introduction](#)
- [2. Code changes](#)
- [3. Results](#)
- [Appendix A. Problems in the original code](#)
- [Appendix B. All timing rules and where they come from](#)
- [Appendix C. Verification](#)
- [Appendix D. Width 1 compared with the original code](#)
- [Appendix E. Comparison with the sample report](#)

This is the report for part 2 of the assignment; the code is in [o3-simulator/tomasulo-python/](#), and paths written as [tests/](#), [results/](#), [verify/](#) or [code/](#) are inside that directory. How to run everything is in [README.md](#). The parts of this report that quote run results are written by [report_tables.py](#) from simulator runs: the paragraph of section 1.4 on one CDB, the summary tables of section 3.2 and the notes under them, the test cases 3.3–3.10 and section 3.11. Every number in these parts is computed by the script, and [run_all.sh](#) fails if the report differs from what the script writes now. The numbers in the rest of the text (the TL;DR, sections 1 and 2, the appendices) were written by hand from the files in [results/](#). "The handout" means the assignment PDF ([programming-tomasulo.pdf](#)), which is not included in this repository; "the sample report" is [Sample_Tomasulo_Assignment_Report.pdf](#) from the course.

1. Introduction

Tomasulo's algorithm lets instructions execute out of order: an instruction waits in a reservation station until its operands arrive on the common data bus (CDB), then executes. Results are kept in the reorder buffer (ROB) and commit in program order, so registers and memory change in program order even when execution does not. The register alias table (RAT) records, for each register, its value or the ROB entry that will produce it.

1.1 What we did

1. Fixed five bugs of the original code, each shown by a small input (Appendix A).
2. **Multiple issue.** Up to W instructions issue per cycle, in program order, $W = 1$ to 4 (`Issue width = W` in the input file, or `--width W`; default 1). Issue stops at the first instruction that finds no free ROB entry, reservation station or load/store queue entry. Up to W results write back and up to W instructions commit per cycle (section 1.4).
3. **Branch unit** (handout pages 1–2): 1-bit predictor and 8-entry BTB, prediction at issue, resolution at the end of EX. After a misprediction found in cycle n , the wrong-path instructions are squashed, the RAT is recovered, and the correct instruction is fetched in cycle $n+2$.
4. **Memory** (handout pages 1–3): 256 bytes; `Ld` and `Sd` move 4-byte single-precision values; addresses are byte addresses. Stores write memory only when they commit. A load forwards from an older store to the same address (1 cycle), and does not wait for older stores whose address is unknown. When such a store turns out to overlap the load, the load and the instructions after it are executed again.
5. **Input and output** in the handout's format: the configuration table, ROB size, issue width, initial register and memory values and the program are read from the input file; the output is the instruction table, all registers and the non-zero memory.
6. 25 test inputs (section 3, Appendix C) and scripts that check the results.

1.2 What works

Issue widths 1–4 with any configuration of reservation stations, FUs and latencies; all ten instructions of the handout (`Ld`, `Sd`, `Beq`, `Bne`, `Add`, `Addi`, `Sub`, `Add.d`, `Sub.d`, `Mult.d`); branch prediction and recovery; forwarding; loads that run ahead of stores and are replayed when needed. All checks of Appendix C pass. We know of no input that gives a wrong result.

1.3 Limitations, and choices the handout leaves open

- **CDB count and commit width** (section 1.4).
- **The memory-order check is our addition.** The handout lets a load go ahead of a store with an unknown address, but gives no way to notice when that store writes the load's address. Without the check such a load keeps an old value (test case 6 shows it).
- **Byte addresses that are not multiples of 4 are allowed**, because the handout's own sample loads from address 18. A 4-byte value at address a uses bytes a to $a+3$, so a can be 0 to 252. Two accesses that share only some bytes cannot forward; the load waits until the store has written memory. Byte order is little-endian (the handout does not say).
- **Arithmetic precision.** `Ld` and `Sd` are single precision, as the handout says. FP registers hold Python floats (double precision), so `Add.d`, `Sub.d` and `Mult.d` compute in double precision, and a value is rounded to single precision when a store writes it. Integer registers are Python integers and do not wrap around at 32 bits.
- The BTB is updated when a branch resolves, also by branches that are later squashed.
- The handout's sample program never ends (`Bne R2, R3, -3` always jumps back). The simulator stops it at `--max-cycles` and says so (test case 1 shows the first 40 cycles).
- Further choices about timing (which instruction goes first, when a freed entry can be used again) are listed with their source in Appendix B.

1.4 One CDB or several?

The handout lists "one CDB" among the processor's components. The sample report extends write back and commit with the issue width: up to W results are broadcast and up to W instructions commit per cycle (its section 2, Code Changes), and its result tables show two results written back in the same cycle.

With only one CDB, at most one result per cycle can be written back. How much that limits a wider issue depends on the program. Our test `wide` has 8 independent instructions on 4 integer adders; at width 4 up to 4 of its results are ready in the same cycle, and with one CDB it takes 11 cycles at every width. `independent` has 6 independent instructions on units with different latencies; with one CDB it still goes from 12 cycles at width 1 to 9 at width 2 (section 3.2).

Our default follows the sample report: the number of CDB buses and the commit width are equal to the issue width. Both can be set separately (`CDB buses = N`, `Commit width = N` in the input, or `--cdb N`, `--commit-width N`). Section 3.2 gives every test in three settings:

Setting	CDB buses	Commit width	What changes with <i>W</i>
default (sample report)	<i>W</i>	<i>W</i>	issue, write back and commit together
one CDB (handout)	1	<i>W</i>	issue and commit
one CDB, one commit	1	1	issue only

The full tables of sections 3.3–3.10 use the default setting. Their cycle counts show what widening issue, write back and commit together gives, not what a wider issue alone gives.

2. Code changes

We kept the original's files, function names, data structure fields and the order in which the stages run in each cycle (issue, exe, mem, wb, commit). Each change below shows the original code ("Initial version") and ours ("Our version"), shortened where marked with `....`. The full difference is `git diff` from the commit that added the original files (see README).

The original code passed 5 to 16 separate arguments to each stage. We pass one object `st` (`State`) that holds the whole machine, and replaced the original's data types (a new `namedtuple` class for every entry) with plain classes.

2.1 Issue up to *W* instructions per cycle (`issue.py`)

The original issued one instruction per cycle and stopped fetching at every `Bne` until it resolved:

Initial version

```
def issue(cycle, PC, instructions, ROB, size_ROB, rs_int_adder, rs_fp_adder, rs_fp_multi,
          ld_sd_queue, size_ld_sd_queue, rat_int, rat_fp):
    # fetch 1 instruction
    ins = instructions[PC.PC]
    ...
    elif (op=='Bne'):
        if (len(ROB)<size_ROB)&(check_rs_space(rs_int_adder)>=0):
            ...
            PC.valid = 0
```

Our version

```
def issue(cycle, st):
    if cycle < st.fetch_resume:          # recovering from a misprediction
        return
    for _ in range(st.issue_width):
        if not (0 <= st.PC < len(st.instructions)):
            return
        ins = st.instructions[st.PC]
        if len(st.ROB) >= st.size_ROB:   # structural hazard: this one and all later ones wait
            return
        ...                               # same for the reservation station / load-store queue
        rob = put_ins_into_ROB(st, cycle, ins)
        ...                               # reservation station or queue, then RAT
        if ins.op in ('Beq', 'Bne'):
            rob.pred_next = predict_branch(st, st.PC)
            st.PC = rob.pred_next
            if rob.pred_next != rob.PC + 1:
                return                    # predicted taken: the target is fetched next cycle
        else:
            st.PC += 1
```

Instructions issued in the same cycle can depend on each other: each one is renamed before the next one reads the RAT.

`predict_branch` looks up BTB entry `PC mod 8`; it predicts taken only if the entry belongs to this branch (it stores the branch's PC) and its bit says the branch was taken last time.

2.2 Write back up to *N* results per cycle (`wb.py`)

The original moved one result per cycle onto its single CDB, and wrote every broadcast value into the RAT, even after a younger instruction had renamed the register (bug B1):

Initial version

```
def wb(cdb, rat_int, rat_fp, rs_int_adder, rs_fp_adder, rs_fp_multi, ld_sd_queue, ROB, cycle,
      results_buffer):
    if cdb.valid == 1:
        broadcast(cdb, ...)
        cdb.valid = 0
    if cdb.valid == 0:
        if len(results_buffer)>0:
            cdb.valid = 1
        ...
        results_buffer.popleft()
# in broadcast:
if reg_tag[0] == 'F':
    rat_fp[int(reg_tag[1:])] = cdb.value
```

Our version

```
def wb(cycle, st):
    waiting = [r for r in st.results_buffer if r.ready < cycle]
    waiting.sort(key=lambda r: (r.ready, r.rob.seq)) # finished first, then oldest
    for result in waiting[:st.cdb_buses]:
        st.results_buffer.remove(result)
        broadcast(st, result.rob, result.value, cycle)
# in broadcast:
if reg_tag[0] == 'F' and st.rat_fp[int(reg_tag[1:])] == dest_tag:
    st.rat_fp[int(reg_tag[1:])] = value
```

2.3 Commit up to W instructions per cycle; stores write memory here (`commit.py`)

The original committed one instruction per cycle (plus one after a store), gave branches no commit cycle, crashed when a branch emptied the ROB (B5), and let stores write memory in the MEM stage as soon as they were first in the load/store queue, before older instructions committed (B2). Now a store writes memory when it commits:

Initial version

```
def commit(ROB, reg_int, reg_fp, cycle, instructions):
    if len(ROB)>0:
        if instructions[ROB[0].PC].split(' ')[0]=='Bne':
            if len(ROB[0].exe)!=0:
                entry = ROB.popleft()
                print_ROB(entry, instructions)
        if (len(ROB)>0)&(instructions[ROB[0].PC].split(' ')[0]!='Bne'): # IndexError when empty
            if len(ROB[0].cdb)!=0: # broadcasted instructions
                ROB[0].commit.append(cycle+1)
        ...
# in mem.py: a store at the queue head goes to memory; this test was always true
if (check_if_sd_committable):
```

Our version

```
def commit(cycle, st):
    for _ in range(st.commit_width):
        if len(st.ROB) == 0:
            return
        head = st.ROB[0]
        if not ready_to_commit(head, st, cycle): # WB (branch: EX) in an earlier cycle;
            return # store: address, data, free memory
        ...
        if head.ins.op == 'Sd':
            write_single(st.memory, address, head.lsqr.data)
            head.mem = [cycle, cycle + st.time_mem - 1]
            st.mem_busy_until = cycle + st.time_mem - 1
            st.ld_sd_queue.remove(head.lsqr)
        modify_arch_reg(head, st)
        head.commit.append(cycle)
        st.ROB.popleft()
```

2.4 Execute: several FUs, oldest first, branch resolution (`exe.py`)

When an instruction that had been waiting entered a pipelined unit, the original called `fu_exe` a second time in the same cycle, which moved every instruction in that unit forward again (bug B4: a 15-cycle multiply took 14). It also ignored the "# of FUs" column and took the ready instruction in the lowest-numbered station, not the oldest:

Initial version

```
if check_valid_ins_in_rs(rs_int_adder)>=0:
    index = check_valid_ins_in_rs(rs_int_adder)
    fu_int_adder.append(fu_entry())
    add_entry_into_fu(fu_int_adder, rs_int_adder[index])
    # check if it's a waiting instruction
    if ROB[find_ROB_entry(ROB, rs_int_adder[index].dest_tag)].issue[0] < cycle:
        fu_exe(fu_int_adder, results_buffer, ROB, time_fu_int_adder, cycle, PC)
```

Our version

```
for kind in ('int', 'fpadd', 'fpmul'):
    for element in check_valid_ins_in_rs(st.rs[kind], cycle): # ready, oldest first
        unit = find_free_fu(st.fu[kind], cycle)
        if unit is None:
            break
        start_exe(st, kind, unit, element.rob, element.value_1st, element.value_2nd, cycle)
        element.busy = 0
```

Each instruction in EX records the cycle its EX ends, and finishes in that cycle; nothing is stepped twice. A pipelined unit can start a new instruction every cycle, an unpipelined one only after the previous instruction's EX ends (`start_exe`). A branch is resolved at the end of its EX (`resolve_branch`): it trains its BTB entry, and if the prediction was wrong, `squash` (section 2.6) removes the wrong path.

2.5 Loads: forwarding, running ahead of stores, memory-order check (`mem.py`)

The original made a load wait until every older store had its address, and could forward to a load in the same cycle in which the load computed its address (bug B3). Our MEM stage, for each load whose address was computed in an earlier cycle:

```
store = forward_check_from_sd(st.ld_sd_queue, index, cycle)
# = the youngest older store whose address is known and that shares a byte
# with the load; stores with no address yet are not considered
if store is None:
    if to_memory is None:
        to_memory = element # read memory when it is free
elif (store.address == element.address) and (store.data_valid == 1):
    element.rob.mem = [cycle, cycle] # forward: 1 cycle
    ...
    st.results_buffer.append(fu_result(element.rob, to_single(store.data), cycle))
# else wait: the data is not there yet, or the store overlaps only partly
```

A load that went ahead of a store with an unknown address read the wrong value if that store overlaps it. In the cycle after a store's address is calculated, `check_memory_order` compares it with the younger loads that already got (or are getting) their data:

```
for rob in st.ROB:
    if (rob.ins.op != 'Ld') or (rob.seq < store.rob.seq) or (len(rob.mem) == 0):
        continue
    if not overlap(store.address, rob.lsq.address):
        continue
    if rob.forwarded and (rob.forwarded_from.seq > store.rob.seq):
        continue # forwarded from a younger store at the same address: still right
    ...         # the oldest such load is the victim
# in mem():
if victim is not None:
    squash(st, victim.seq - 1, victim.PC, cycle) # the load is squashed too
```

2.6 Recovery (`squash.py`, new)

Called for a mispredicted branch (keep the branch, fetch its correct target) and for a memory-order violation (keep what is before the load, fetch the load again). It does the handout's three actions and removes the wrong-path instructions from every other place they can be:

```
def squash(st, last_kept_seq, next_pc, cycle):
    # 1. ROB: drop every entry younger than last_kept_seq
    # 2. their reservation stations (and wait-for tags), load/store queue entries,
    #    instructions in the FUs, results waiting for a CDB, a load using the memory
    # 3. RAT: start from the architectural registers, then walk the remaining ROB
    #    entries oldest to youngest: a register maps to its youngest producer
    #    (its value if already broadcast, else its tag)
    st.PC = next_pc
    st.fetch_resume = cycle + 2 # handout: detected in cycle n, fetch in n+2
```

Rebuilding the RAT from the ROB needs no saved copies. Every remaining ROB entry is older than the squashed instructions. For each register, the youngest remaining entry that writes it is the producer the RAT pointed to before the squashed instructions were issued. If that producer has broadcast since, we store its value; if it has committed since, the architectural register holds its value.

2.7 Memory: 256 bytes, single precision (`init.py`)

The original's memory was a list of 256 slots, each holding a whole Python number. Ours is 256 bytes; `write_single` stores the 4-byte single-precision encoding of a value at its byte address, and loads decode the same 4 bytes. Forwarding uses `to_single` too, so a load gets the same value whether it reads memory or forwards from a store.

2.8 Input file and output (`read_input`, `print_status.py`)

The original's configuration was fixed in `main.py`, the program came from `code.in`, and `test_case.txt` was never read; it knew only `Bne` with a byte offset, no commas, and printed no registers or memory. Now everything comes from the input file in the handout's format (README). At the end we print all registers and the non-zero memory words, each value with the fewest digits that read back as the same number. The run ends only when nothing is left to fetch, the ROB is empty and the last store has finished writing (`finished`); the original stopped as soon as the ROB was empty, before the last store's write (B3).

3. Results

3.1 How to read the tables

One row per committed instruction, in program order; an instruction run several times by a loop appears once per run. Squashed (wrong-path) instructions are not listed. Cycles are written as in the original code: `[first, last]` for EX and MEM, `[cycle]` for ISSUE, WB and COMMIT, `[]` when the instruction has no such stage.

- EX of a load or store is its address calculation. MEM of a load is the memory read, or one cycle when the value is forwarded from a store. MEM of a store is its write, which starts in the cycle it commits.

- NOTE: **taken** / **not taken** and **mispredicted** for branches; **forwarded** for a load that got its value from a store; **replayed** for a load executed again after a memory-order violation.
- Cycles = the last cycle in which anything happened (the last commit, or the end of the last store's write). IPC = committed instructions / cycles.

3.2 Summary of all tests

Default setting: CDB buses and commit width equal to the issue width (unless the input file sets them: **cdb_limit** and **squash_cdb** have one CDB).

Test	Instructions	Cycles W=1	W=2	W=3	W=4	IPC W=1	W=2	W=3	W=4	Mispredicted branches	Memory- order violations	Squashed W=1 / W=4
btb_alias	4	10	10	10	10	0.40	0.40	0.40	0.40	1	0	2 / 4
bug1_rat_overwrite	7	37	36	36	36	0.19	0.19	0.19	0.19	0	0	0 / 0
bug2_store_before_commit	3	23	23	23	23	0.13	0.13	0.13	0.13	0	0	0 / 0
bug3_forward_same_cycle	3	14	13	13	13	0.21	0.23	0.23	0.23	0	0	0 / 0
bug4_double_advance	3	26	25	25	25	0.12	0.12	0.12	0.12	0	0	0 / 0
bug5_empty_rob_crash	7	20	20	20	20	0.35	0.35	0.35	0.35	2	0	4 / 4
cdb_limit	8	11	11	11	11	0.73	0.73	0.73	0.73	0	0	0 / 0
chain	6	22	22	22	22	0.27	0.27	0.27	0.27	0	0	0 / 0
daxpy	28	41	32	31	31	0.68	0.88	0.90	0.90	2	0	2 / 4
dispatch_order	4	8	8	8	8	0.50	0.50	0.50	0.50	0	0	0 / 0
independent	6	12	9	8	8	0.50	0.67	0.75	0.75	0	0	0 / 0
loop	6	16	16	16	16	0.38	0.38	0.38	0.38	2	0	4 / 5
memory	7	16	14	14	14	0.44	0.50	0.50	0.50	0	0	0 / 0
memory_violation	4	18	18	18	18	0.22	0.22	0.22	0.22	0	1	2 / 2
original	15	41	35	34	34	0.37	0.43	0.44	0.44	2	0	2 / 4
precision	7	14	13	13	13	0.50	0.54	0.54	0.54	0	0	0 / 0
r0	6	12	10	10	10	0.50	0.60	0.60	0.60	1	0	2 / 2
sample_tc1	17	36	33	33	33	0.47	0.52	0.52	0.52	0	0	0 / 0
sample_tc2	17	49	47	47	47	0.35	0.36	0.36	0.36	0	0	0 / 0
squash_cdb	6	14	12	12	11	0.43	0.50	0.50	0.55	1	0	3 / 3
structural	8	20	19	18	18	0.40	0.42	0.44	0.44	0	0	0 / 0
unaligned	7	17	15	15	15	0.41	0.47	0.47	0.47	0	0	0 / 0
violation_forward	5	17	16	15	15	0.29	0.31	0.33	0.33	0	1	1 / 1
wide	8	11	7	6	5	0.73	1.14	1.33	1.60	0	0	0 / 0

One CDB, as the handout lists: cycles with one CDB bus and the commit width equal to the issue width, and with one CDB bus and one commit per cycle (then only the issue width changes).

Test	One CDB: W=1	W=2	W=3	W=4	One CDB, one commit: W=1	W=2	W=3	W=4
btb_alias	10	10	10	10		10	10	10
bug1_rat_overwrite	37	36	36	36		37	37	37
bug2_store_before_commit	23	23	23	23		23	23	23
bug3_forward_same_cycle	14	13	13	13		14	14	14
bug4_double_advance	26	25	25	25		26	25	25
bug5_empty_rob_crash	20	20	20	20		20	20	20
cdb_limit	11	11	11	11		11	11	11
chain	22	22	22	22		22	22	22

Test	One CDB: W=1	W=2	W=3	W=4	One CDB, one commit: W=1	W=2	W=3	W=4
daxpy	41	34	33	33	41	41	41	41
dispatch_order	8	8	8	8	8	8	8	8
independent	12	9	9	9	12	10	10	10
loop	16	16	16	16	16	16	16	16
memory	16	14	14	14	16	15	15	15
memory_violation	18	18	18	18	18	18	18	18
original	41	35	34	34	41	41	41	41
precision	14	13	13	13	14	13	13	13
r0	12	10	10	10	12	10	10	10
sample_tc1	36	35	35	35	36	36	36	36
sample_tc2	49	47	47	47	49	47	47	47
squash_cdb	14	12	12	11	14	12	12	11
structural	20	19	18	18	20	20	20	20
unaligned	17	15	15	15	17	17	17	17
violation_forward	17	17	16	16	17	17	16	16
wide	11	11	11	11	11	11	11	11

What the summary shows (details in section 3.11):

- With the default setting, a wider issue helps programs with independent work (from width 1 to 4: **wide** 11 → 5, **independent** 12 → 8, **daxpy** 41 → 31, **original** 41 → 34 cycles) and does nothing for a dependence chain (**chain**, 22 cycles at every width) or for a loop whose time is decided by its 2 mispredictions (**loop**, 16 cycles at every width).
- With one CDB, **wide** stays at 11 cycles at every width: at width 4 its 8 results are written back one per cycle, in cycles 3–10. Other programs still gain from a wider issue: **independent** 12 → 9, **daxpy** 41 → 33, **original** 41 → 34. In **independent** the results are ready in cycles 2–7 at width 2, so the single CDB writes one back in each of cycles 3–8; at width 1 it writes back only in cycles 3, 4, 6, 7, 8 and 11.
- With one CDB and one commit per cycle, 16 of the 24 programs that finish take the same number of cycles at every width, among them **daxpy** and **original** (41 cycles each); for **daxpy** the limit is commit (section 3.11). The other 8 end 1 to 3 cycles earlier at width 4 than at width 1, for example **independent** 12 → 10.

3.3 Test case 1: pdf_sample

The sample input of the assignment handout (page 3), unchanged. Bne R2, R3, -3 jumps back to the first instruction and R2, R3 never change, so the program never ends; check_all.py stops it after 200 cycles.

Configuration and initial values (tests/pdf_sample.txt):

```

# of rs  Cycles in EX  Cycles in Mem  # of FUs
Integer adder  2      1              1
FP adder      3      3              1
FP multiplier  2     20              1
Load/store unit 3      4              1
ROB entries = 128
Issue width = 2

R1=10, R2=20, F2=30.1
Mem[4]=1, Mem[8]=2, Mem[12]=3.4

```

Instructions (the number is the instruction index that branch offsets count from):

```

0  Add.d F1, F2, F3
1  Ld F4, 8(R1)
2  Bne R2, R3, -3

```

Figure 1. pdf_sample, issue width 1 (CDB buses 1, commit width 1): 40 cycles, 25 committed instructions, IPC 0.62, 0 squashed. Stopped by the 40-cycle limit.

	ISSUE	EX	MEM	WB	COMMIT	NOTE
Add.d F1, F2, F3	[1]	[2, 4]	[]	[5]	[6]	
Ld F4, 8(R1)	[2]	[3, 3]	[4, 7]	[8]	[9]	
Bne R2, R3, -3	[3]	[4, 4]	[]	[]	[10]	taken, mispredicted
Add.d F1, F2, F3	[6]	[7, 9]	[]	[10]	[11]	
Ld F4, 8(R1)	[7]	[8, 8]	[9, 12]	[13]	[14]	
Bne R2, R3, -3	[8]	[9, 9]	[]	[]	[15]	taken
Add.d F1, F2, F3	[9]	[10, 12]	[]	[14]	[16]	
Ld F4, 8(R1)	[10]	[11, 11]	[13, 16]	[17]	[18]	
Bne R2, R3, -3	[11]	[12, 12]	[]	[]	[19]	taken
Add.d F1, F2, F3	[12]	[13, 15]	[]	[16]	[20]	
Ld F4, 8(R1)	[13]	[14, 14]	[17, 20]	[21]	[22]	
Bne R2, R3, -3	[14]	[15, 15]	[]	[]	[23]	taken
Add.d F1, F2, F3	[15]	[16, 18]	[]	[19]	[24]	
Ld F4, 8(R1)	[16]	[17, 17]	[21, 24]	[25]	[26]	
Bne R2, R3, -3	[17]	[18, 18]	[]	[]	[27]	taken
Add.d F1, F2, F3	[18]	[19, 21]	[]	[22]	[28]	
Ld F4, 8(R1)	[19]	[20, 20]	[25, 28]	[29]	[30]	
Bne R2, R3, -3	[20]	[21, 21]	[]	[]	[31]	taken
Add.d F1, F2, F3	[21]	[22, 24]	[]	[26]	[32]	
Ld F4, 8(R1)	[22]	[23, 23]	[29, 32]	[33]	[34]	
Bne R2, R3, -3	[23]	[24, 24]	[]	[]	[35]	taken
Add.d F1, F2, F3	[24]	[25, 27]	[]	[28]	[36]	
Ld F4, 8(R1)	[25]	[26, 26]	[33, 36]	[37]	[38]	
Bne R2, R3, -3	[26]	[27, 27]	[]	[]	[39]	taken
Add.d F1, F2, F3	[27]	[28, 30]	[]	[31]	[40]	

Figure 2. pdf_sample, issue width 2 (CDB buses 2, commit width 2): 40 cycles, 27 committed instructions, IPC 0.68, 0 squashed. Stopped by the 40-cycle limit.

	ISSUE	EX	MEM	WB	COMMIT	NOTE
Add.d F1, F2, F3	[1]	[2, 4]	[]	[5]	[6]	
Ld F4, 8(R1)	[1]	[2, 2]	[3, 6]	[7]	[8]	
Bne R2, R3, -3	[2]	[3, 3]	[]	[]	[8]	taken, mispredicted
Add.d F1, F2, F3	[5]	[6, 8]	[]	[9]	[10]	
Ld F4, 8(R1)	[5]	[6, 6]	[7, 10]	[11]	[12]	
Bne R2, R3, -3	[6]	[7, 7]	[]	[]	[12]	taken
Add.d F1, F2, F3	[7]	[8, 10]	[]	[11]	[13]	
Ld F4, 8(R1)	[7]	[8, 8]	[11, 14]	[15]	[16]	
Bne R2, R3, -3	[8]	[9, 9]	[]	[]	[16]	taken
Add.d F1, F2, F3	[9]	[10, 12]	[]	[13]	[17]	
Ld F4, 8(R1)	[9]	[10, 10]	[15, 18]	[19]	[20]	
Bne R2, R3, -3	[10]	[11, 11]	[]	[]	[20]	taken
Add.d F1, F2, F3	[11]	[12, 14]	[]	[15]	[21]	
Ld F4, 8(R1)	[11]	[12, 12]	[19, 22]	[23]	[24]	
Bne R2, R3, -3	[12]	[13, 13]	[]	[]	[24]	taken
Add.d F1, F2, F3	[13]	[14, 16]	[]	[17]	[25]	
Ld F4, 8(R1)	[15]	[16, 16]	[23, 26]	[27]	[28]	
Bne R2, R3, -3	[15]	[16, 16]	[]	[]	[28]	taken
Add.d F1, F2, F3	[16]	[17, 19]	[]	[20]	[29]	
Ld F4, 8(R1)	[19]	[20, 20]	[27, 30]	[31]	[32]	
Bne R2, R3, -3	[19]	[20, 20]	[]	[]	[32]	taken
Add.d F1, F2, F3	[20]	[21, 23]	[]	[24]	[33]	
Ld F4, 8(R1)	[23]	[24, 24]	[31, 34]	[35]	[36]	
Bne R2, R3, -3	[23]	[24, 24]	[]	[]	[36]	taken
Add.d F1, F2, F3	[24]	[25, 27]	[]	[28]	[37]	
Ld F4, 8(R1)	[27]	[28, 28]	[35, 38]	[39]	[40]	
Bne R2, R3, -3	[27]	[28, 28]	[]	[]	[40]	taken

Figure 3. pdf_sample, issue width 3 (CDB buses 3, commit width 3): 40 cycles, 28 committed instructions, IPC 0.70, 0 squashed. Stopped by the 40-cycle limit.

	ISSUE	EX	MEM	WB	COMMIT	NOTE
Add.d F1, F2, F3	[1]	[2, 4]	[]	[5]	[6]	
Ld F4, 8(R1)	[1]	[2, 2]	[3, 6]	[7]	[8]	
Bne R2, R3, -3	[1]	[2, 2]	[]	[]	[8]	taken, mispredicted
Add.d F1, F2, F3	[4]	[5, 7]	[]	[8]	[9]	
Ld F4, 8(R1)	[4]	[5, 5]	[7, 10]	[11]	[12]	
Bne R2, R3, -3	[4]	[5, 5]	[]	[]	[12]	taken
Add.d F1, F2, F3	[5]	[6, 8]	[]	[9]	[12]	
Ld F4, 8(R1)	[5]	[6, 6]	[11, 14]	[15]	[16]	
Bne R2, R3, -3	[5]	[6, 6]	[]	[]	[16]	taken
Add.d F1, F2, F3	[6]	[7, 9]	[]	[10]	[16]	
Ld F4, 8(R1)	[7]	[8, 8]	[15, 18]	[19]	[20]	
Bne R2, R3, -3	[7]	[8, 8]	[]	[]	[20]	taken
Add.d F1, F2, F3	[8]	[9, 11]	[]	[12]	[20]	
Ld F4, 8(R1)	[11]	[12, 12]	[19, 22]	[23]	[24]	
Bne R2, R3, -3	[11]	[12, 12]	[]	[]	[24]	taken
Add.d F1, F2, F3	[12]	[13, 15]	[]	[16]	[24]	
Ld F4, 8(R1)	[15]	[16, 16]	[23, 26]	[27]	[28]	
Bne R2, R3, -3	[15]	[16, 16]	[]	[]	[28]	taken
Add.d F1, F2, F3	[16]	[17, 19]	[]	[20]	[28]	
Ld F4, 8(R1)	[19]	[20, 20]	[27, 30]	[31]	[32]	
Bne R2, R3, -3	[19]	[20, 20]	[]	[]	[32]	taken
Add.d F1, F2, F3	[20]	[21, 23]	[]	[24]	[32]	
Ld F4, 8(R1)	[23]	[24, 24]	[31, 34]	[35]	[36]	
Bne R2, R3, -3	[23]	[24, 24]	[]	[]	[36]	taken
Add.d F1, F2, F3	[24]	[25, 27]	[]	[28]	[36]	
Ld F4, 8(R1)	[27]	[28, 28]	[35, 38]	[39]	[40]	
Bne R2, R3, -3	[27]	[28, 28]	[]	[]	[40]	taken
Add.d F1, F2, F3	[28]	[29, 31]	[]	[32]	[40]	

Figure 4. pdf_sample, issue width 4 (CDB buses 4, commit width 4): 40 cycles, 28 committed instructions, IPC 0.70, 0 squashed. Stopped by the 40-cycle limit.

	ISSUE	EX	MEM	WB	COMMIT	NOTE
Add.d F1, F2, F3	[1]	[2, 4]	[]	[5]	[6]	
Ld F4, 8(R1)	[1]	[2, 2]	[3, 6]	[7]	[8]	
Bne R2, R3, -3	[1]	[2, 2]	[]	[]	[8]	taken, mispredicted
Add.d F1, F2, F3	[4]	[5, 7]	[]	[8]	[9]	
Ld F4, 8(R1)	[4]	[5, 5]	[7, 10]	[11]	[12]	
Bne R2, R3, -3	[4]	[5, 5]	[]	[]	[12]	taken
Add.d F1, F2, F3	[5]	[6, 8]	[]	[9]	[12]	
Ld F4, 8(R1)	[5]	[6, 6]	[11, 14]	[15]	[16]	
Bne R2, R3, -3	[5]	[6, 6]	[]	[]	[16]	taken
Add.d F1, F2, F3	[6]	[7, 9]	[]	[10]	[16]	
Ld F4, 8(R1)	[7]	[8, 8]	[15, 18]	[19]	[20]	
Bne R2, R3, -3	[7]	[8, 8]	[]	[]	[20]	taken
Add.d F1, F2, F3	[8]	[9, 11]	[]	[12]	[20]	
Ld F4, 8(R1)	[11]	[12, 12]	[19, 22]	[23]	[24]	
Bne R2, R3, -3	[11]	[12, 12]	[]	[]	[24]	taken
Add.d F1, F2, F3	[12]	[13, 15]	[]	[16]	[24]	
Ld F4, 8(R1)	[15]	[16, 16]	[23, 26]	[27]	[28]	
Bne R2, R3, -3	[15]	[16, 16]	[]	[]	[28]	taken
Add.d F1, F2, F3	[16]	[17, 19]	[]	[20]	[28]	
Ld F4, 8(R1)	[19]	[20, 20]	[27, 30]	[31]	[32]	
Bne R2, R3, -3	[19]	[20, 20]	[]	[]	[32]	taken
Add.d F1, F2, F3	[20]	[21, 23]	[]	[24]	[32]	
Ld F4, 8(R1)	[23]	[24, 24]	[31, 34]	[35]	[36]	
Bne R2, R3, -3	[23]	[24, 24]	[]	[]	[36]	taken
Add.d F1, F2, F3	[24]	[25, 27]	[]	[28]	[36]	
Ld F4, 8(R1)	[27]	[28, 28]	[35, 38]	[39]	[40]	
Bne R2, R3, -3	[27]	[28, 28]	[]	[]	[40]	taken
Add.d F1, F2, F3	[28]	[29, 31]	[]	[32]	[40]	

3.4 Test case 2: sample_tc1

Test case #01 of the sample report (Sample_Tomasulo_Assignment_Report.pdf), unchanged.

Configuration and initial values ([tests/sample_tc1.txt](#)):

	# of rs	Cycles in EX	Cycles in Mem	# of FUs
Integer adder	4	1		1
FP adder	4	3		1
FP multiplier	4	5		1
Load/store unit	4	1	6	1

ROB entries = 64

R1=12, R2=32, F20=3.0
 Mem[4]=3.0, Mem[8]=2.0, Mem[12]=1.0, Mem[24]=6.0, Mem[28]=5.0, Mem[32]=4.0

Instructions (the number is the instruction index that branch offsets count from):

```

0  Ld F1 0(R1)
1  Mult.d F2 F1 F0
2  Add.d F3 F3 F2
3  Mult.d F2 F1 F1
4  Add.d F4 F2 F4
5  Addi R1 R1 8
6  Ld F1 0(R1)
7  Mult.d F2 F1 F0
8  Add.d F3 F3 F2
9  Mult.d F2 F1 F1
10 Add.d F4 F2 F4
11 Addi R1 R1 8
12 Ld F1 0(R1)
13 Mult.d F2 F1 F0
14 Add.d F3 F3 F2
15 Mult.d F2 F1 F1
16 Add.d F4 F2 F4

```

Final non-zero registers and memory words (the same at every width): R1 = 28, R2 = 32, F1 = 5.0, F2 = 25.0, F4 = 26.0, F20 = 3.0, Mem[4] = 3.0, Mem[8] = 2.0, Mem[12] = 1.0, Mem[24] = 6.0, Mem[28] = 5.0, Mem[32] = 4.0.

Figure 5. [sample_tc1](#), issue width 1 (CDB buses 1, commit width 1): 36 cycles, 17 committed instructions, IPC 0.47, 0 squashed.

	ISSUE	EX	MEM	WB	COMMIT	NOTE
Ld F1 0(R1)	[1]	[2, 2]	[3, 8]	[9]	[10]	
Mult.d F2 F1 F0	[2]	[10, 14]	[]	[15]	[16]	
Add.d F3 F3 F2	[3]	[16, 18]	[]	[19]	[20]	
Mult.d F2 F1 F1	[4]	[11, 15]	[]	[16]	[21]	
Add.d F4 F2 F4	[5]	[17, 19]	[]	[20]	[22]	
Addi R1 R1 8	[6]	[7, 7]	[]	[8]	[23]	
Ld F1 0(R1)	[7]	[9, 9]	[10, 15]	[17]	[24]	
Mult.d F2 F1 F0	[8]	[18, 22]	[]	[23]	[25]	
Add.d F3 F3 F2	[9]	[24, 26]	[]	[27]	[28]	
Mult.d F2 F1 F1	[10]	[19, 23]	[]	[24]	[29]	
Add.d F4 F2 F4	[11]	[25, 27]	[]	[28]	[30]	
Addi R1 R1 8	[12]	[13, 13]	[]	[14]	[31]	
Ld F1 0(R1)	[13]	[15, 15]	[16, 21]	[22]	[32]	
Mult.d F2 F1 F0	[14]	[23, 27]	[]	[29]	[33]	
Add.d F3 F3 F2	[17]	[30, 32]	[]	[33]	[34]	
Mult.d F2 F1 F1	[18]	[24, 28]	[]	[30]	[35]	
Add.d F4 F2 F4	[19]	[31, 33]	[]	[34]	[36]	

Figure 6. `sample_tc1`, issue width 2 (CDB buses 2, commit width 2): 33 cycles, 17 committed instructions, IPC 0.52, 0 squashed.

	ISSUE	EX	MEM	WB	COMMIT	NOTE
Ld F1 0(R1)	[1]	[2, 2]	[3, 8]	[9]	[10]	
Mult.d F2 F1 F0	[1]	[10, 14]	[]	[15]	[16]	
Add.d F3 F3 F2	[2]	[16, 18]	[]	[19]	[20]	
Mult.d F2 F1 F1	[2]	[11, 15]	[]	[16]	[20]	
Add.d F4 F2 F4	[3]	[17, 19]	[]	[20]	[21]	
Addi R1 R1 8	[3]	[4, 4]	[]	[5]	[21]	
Ld F1 0(R1)	[4]	[6, 6]	[9, 14]	[15]	[22]	
Mult.d F2 F1 F0	[4]	[16, 20]	[]	[21]	[22]	
Add.d F3 F3 F2	[5]	[22, 24]	[]	[25]	[26]	
Mult.d F2 F1 F1	[5]	[17, 21]	[]	[22]	[26]	
Add.d F4 F2 F4	[6]	[23, 25]	[]	[26]	[27]	
Addi R1 R1 8	[6]	[7, 7]	[]	[8]	[27]	
Ld F1 0(R1)	[7]	[9, 9]	[15, 20]	[21]	[28]	
Mult.d F2 F1 F0	[11]	[22, 26]	[]	[27]	[28]	
Add.d F3 F3 F2	[17]	[28, 30]	[]	[31]	[32]	
Mult.d F2 F1 F1	[17]	[23, 27]	[]	[28]	[32]	
Add.d F4 F2 F4	[18]	[29, 31]	[]	[32]	[33]	

Figure 7. `sample_tc1`, issue width 3 (CDB buses 3, commit width 3): 33 cycles, 17 committed instructions, IPC 0.52, 0 squashed.

	ISSUE	EX	MEM	WB	COMMIT	NOTE
Ld F1 0(R1)	[1]	[2, 2]	[3, 8]	[9]	[10]	
Mult.d F2 F1 F0	[1]	[10, 14]	[]	[15]	[16]	
Add.d F3 F3 F2	[1]	[16, 18]	[]	[19]	[20]	
Mult.d F2 F1 F1	[2]	[11, 15]	[]	[16]	[20]	
Add.d F4 F2 F4	[2]	[17, 19]	[]	[20]	[21]	
Addi R1 R1 8	[2]	[3, 3]	[]	[4]	[21]	
Ld F1 0(R1)	[3]	[5, 5]	[9, 14]	[15]	[21]	
Mult.d F2 F1 F0	[3]	[16, 20]	[]	[21]	[22]	
Add.d F3 F3 F2	[3]	[22, 24]	[]	[25]	[26]	
Mult.d F2 F1 F1	[4]	[17, 21]	[]	[22]	[26]	
Add.d F4 F2 F4	[4]	[23, 25]	[]	[26]	[27]	
Addi R1 R1 8	[4]	[5, 5]	[]	[6]	[27]	
Ld F1 0(R1)	[5]	[7, 7]	[15, 20]	[21]	[27]	
Mult.d F2 F1 F0	[11]	[22, 26]	[]	[27]	[28]	
Add.d F3 F3 F2	[17]	[28, 30]	[]	[31]	[32]	
Mult.d F2 F1 F1	[17]	[23, 27]	[]	[28]	[32]	
Add.d F4 F2 F4	[18]	[29, 31]	[]	[32]	[33]	

Figure 8. `sample_tc1`, issue width 4 (CDB buses 4, commit width 4): 33 cycles, 17 committed instructions, IPC 0.52, 0 squashed.

	ISSUE	EX	MEM	WB	COMMIT	NOTE
Ld F1 0(R1)	[1]	[2, 2]	[3, 8]	[9]	[10]	
Mult.d F2 F1 F0	[1]	[10, 14]	[]	[15]	[16]	
Add.d F3 F3 F2	[1]	[16, 18]	[]	[19]	[20]	
Mult.d F2 F1 F1	[1]	[11, 15]	[]	[16]	[20]	
Add.d F4 F2 F4	[2]	[17, 19]	[]	[20]	[21]	
Addi R1 R1 8	[2]	[3, 3]	[]	[4]	[21]	
Ld F1 0(R1)	[2]	[5, 5]	[9, 14]	[15]	[21]	
Mult.d F2 F1 F0	[2]	[16, 20]	[]	[21]	[22]	
Add.d F3 F3 F2	[3]	[22, 24]	[]	[25]	[26]	
Mult.d F2 F1 F1	[3]	[17, 21]	[]	[22]	[26]	
Add.d F4 F2 F4	[3]	[23, 25]	[]	[26]	[27]	
Addi R1 R1 8	[3]	[5, 5]	[]	[6]	[27]	
Ld F1 0(R1)	[4]	[7, 7]	[15, 20]	[21]	[27]	
Mult.d F2 F1 F0	[11]	[22, 26]	[]	[27]	[28]	
Add.d F3 F3 F2	[17]	[28, 30]	[]	[31]	[32]	
Mult.d F2 F1 F1	[17]	[23, 27]	[]	[28]	[32]	
Add.d F4 F2 F4	[18]	[29, 31]	[]	[32]	[33]	

3.5 Test case 3: `sample_tc2`

Test case #02 of the sample report (Sample_Tomasulo_Assignment_Report.pdf), unchanged.

Configuration and initial values ([tests/sample_tc2.txt](#)):

	# of rs	Cycles in EX	Cycles in Mem	# of FUs
Integer adder	4	1		1
FP adder	3	3		2
FP multiplier	2	15		2
Load/store unit	4	1	5	2

R0B entries = 128

R1=12, R2=32, F20=3.0
 Mem[4]=3.0, Mem[8]=2.0, Mem[12]=1.0, Mem[24]=6.0, Mem[28]=5.0, Mem[32]=4.0

Instructions (the number is the instruction index that branch offsets count from):

```

0  Ld F1 0(R1)
1  Mult.d F2 F1 F0
2  Add.d F3 F3 F2
3  Mult.d F2 F1 F1
4  Ld F1 0(R1)
5  Addi R1 R1 4
6  Add.d F4 F2 F4
7  Addi R1 R1 8
8  Ld F1 0(R1)
9  Mult.d F2 F1 F0
10 Add.d F3 F3 F2
11 Mult.d F2 F1 F1
12 Ld F1 0(R1)
13 Mult.d F2 F1 F0
14 Add.d F3 F3 F2
15 Mult.d F2 F1 F1
16 Add.d F4 F2 F4

```

Final non-zero registers and memory words (the same at every width): R1 = 24, R2 = 32, F1 = 6.0, F2 = 36.0, F4 = 37.0, F20 = 3.0, Mem[4] = 3.0, Mem[8] = 2.0, Mem[12] = 1.0, Mem[24] = 6.0, Mem[28] = 5.0, Mem[32] = 4.0.

Figure 9. [sample_tc2](#), issue width 1 (CDB buses 1, commit width 1): 49 cycles, 17 committed instructions, IPC 0.35, 0 squashed.

	ISSUE	EX	MEM	WB	COMMIT	NOTE
Ld F1 0(R1)	[1]	[2, 2]	[3, 7]	[8]	[9]	
Mult.d F2 F1 F0	[2]	[9, 23]	[]	[24]	[25]	
Add.d F3 F3 F2	[3]	[25, 27]	[]	[28]	[29]	
Mult.d F2 F1 F1	[4]	[9, 23]	[]	[25]	[30]	
Ld F1 0(R1)	[5]	[6, 6]	[8, 12]	[13]	[31]	
Addi R1 R1 4	[6]	[7, 7]	[]	[9]	[32]	
Add.d F4 F2 F4	[7]	[26, 28]	[]	[29]	[33]	
Addi R1 R1 8	[8]	[10, 10]	[]	[11]	[34]	
Ld F1 0(R1)	[9]	[12, 12]	[13, 17]	[18]	[35]	
Mult.d F2 F1 F0	[10]	[19, 33]	[]	[34]	[36]	
Add.d F3 F3 F2	[11]	[35, 37]	[]	[38]	[39]	
Mult.d F2 F1 F1	[12]	[19, 33]	[]	[35]	[40]	
Ld F1 0(R1)	[13]	[14, 14]	[18, 22]	[23]	[41]	
Mult.d F2 F1 F0	[20]	[24, 38]	[]	[39]	[42]	
Add.d F3 F3 F2	[26]	[40, 42]	[]	[43]	[44]	
Mult.d F2 F1 F1	[27]	[28, 42]	[]	[44]	[45]	
Add.d F4 F2 F4	[28]	[45, 47]	[]	[48]	[49]	

Figure 10. `sample_tc2`, issue width 2 (CDB buses 2, commit width 2): 47 cycles, 17 committed instructions, IPC 0.36, 0 squashed.

	ISSUE	EX	MEM	WB	COMMIT	NOTE
Ld F1 0(R1)	[1]	[2, 2]	[3, 7]	[8]	[9]	
Mult.d F2 F1 F0	[1]	[9, 23]	[]	[24]	[25]	
Add.d F3 F3 F2	[2]	[25, 27]	[]	[28]	[29]	
Mult.d F2 F1 F1	[2]	[9, 23]	[]	[24]	[29]	
Ld F1 0(R1)	[3]	[4, 4]	[8, 12]	[13]	[30]	
Addi R1 R1 4	[3]	[4, 4]	[]	[5]	[30]	
Add.d F4 F2 F4	[4]	[25, 27]	[]	[28]	[31]	
Addi R1 R1 8	[4]	[6, 6]	[]	[7]	[31]	
Ld F1 0(R1)	[5]	[8, 8]	[13, 17]	[18]	[32]	
Mult.d F2 F1 F0	[10]	[19, 33]	[]	[34]	[35]	
Add.d F3 F3 F2	[10]	[35, 37]	[]	[38]	[39]	
Mult.d F2 F1 F1	[11]	[19, 33]	[]	[34]	[39]	
Ld F1 0(R1)	[11]	[12, 12]	[18, 22]	[23]	[40]	
Mult.d F2 F1 F0	[20]	[24, 38]	[]	[39]	[40]	
Add.d F3 F3 F2	[26]	[40, 42]	[]	[43]	[44]	
Mult.d F2 F1 F1	[26]	[27, 41]	[]	[42]	[44]	
Add.d F4 F2 F4	[27]	[43, 45]	[]	[46]	[47]	

Figure 11. `sample_tc2`, issue width 3 (CDB buses 3, commit width 3): 47 cycles, 17 committed instructions, IPC 0.36, 0 squashed.

	ISSUE	EX	MEM	WB	COMMIT	NOTE
Ld F1 0(R1)	[1]	[2, 2]	[3, 7]	[8]	[9]	
Mult.d F2 F1 F0	[1]	[9, 23]	[]	[24]	[25]	
Add.d F3 F3 F2	[1]	[25, 27]	[]	[28]	[29]	
Mult.d F2 F1 F1	[2]	[9, 23]	[]	[24]	[29]	
Ld F1 0(R1)	[2]	[3, 3]	[8, 12]	[13]	[29]	
Addi R1 R1 4	[2]	[3, 3]	[]	[4]	[30]	
Add.d F4 F2 F4	[3]	[25, 27]	[]	[28]	[30]	
Addi R1 R1 8	[3]	[5, 5]	[]	[6]	[30]	
Ld F1 0(R1)	[3]	[7, 7]	[13, 17]	[18]	[31]	
Mult.d F2 F1 F0	[10]	[19, 33]	[]	[34]	[35]	
Add.d F3 F3 F2	[10]	[35, 37]	[]	[38]	[39]	
Mult.d F2 F1 F1	[10]	[19, 33]	[]	[34]	[39]	
Ld F1 0(R1)	[11]	[12, 12]	[18, 22]	[23]	[39]	
Mult.d F2 F1 F0	[20]	[24, 38]	[]	[39]	[40]	
Add.d F3 F3 F2	[26]	[40, 42]	[]	[43]	[44]	
Mult.d F2 F1 F1	[26]	[27, 41]	[]	[42]	[44]	
Add.d F4 F2 F4	[26]	[43, 45]	[]	[46]	[47]	

Figure 12. `sample_tc2`, issue width 4 (CDB buses 4, commit width 4): 47 cycles, 17 committed instructions, IPC 0.36, 0 squashed.

	ISSUE	EX	MEM	WB	COMMIT	NOTE
Ld F1 0(R1)	[1]	[2, 2]	[3, 7]	[8]	[9]	
Mult.d F2 F1 F0	[1]	[9, 23]	[]	[24]	[25]	
Add.d F3 F3 F2	[1]	[25, 27]	[]	[28]	[29]	
Mult.d F2 F1 F1	[1]	[9, 23]	[]	[24]	[29]	
Ld F1 0(R1)	[2]	[3, 3]	[8, 12]	[13]	[29]	
Addi R1 R1 4	[2]	[3, 3]	[]	[4]	[29]	
Add.d F4 F2 F4	[2]	[25, 27]	[]	[28]	[30]	
Addi R1 R1 8	[2]	[5, 5]	[]	[6]	[30]	
Ld F1 0(R1)	[3]	[7, 7]	[13, 17]	[18]	[30]	
Mult.d F2 F1 F0	[10]	[19, 33]	[]	[34]	[35]	
Add.d F3 F3 F2	[10]	[35, 37]	[]	[38]	[39]	
Mult.d F2 F1 F1	[10]	[19, 33]	[]	[34]	[39]	
Ld F1 0(R1)	[10]	[11, 11]	[18, 22]	[23]	[39]	
Mult.d F2 F1 F0	[20]	[24, 38]	[]	[39]	[40]	
Add.d F3 F3 F2	[26]	[40, 42]	[]	[43]	[44]	
Mult.d F2 F1 F1	[26]	[27, 41]	[]	[42]	[44]	
Add.d F4 F2 F4	[26]	[43, 45]	[]	[46]	[47]	

3.6 Test case 4: `wide`

Eight independent integer instructions and four integer adders: with issue width N, N instructions issue, execute, write back and commit per cycle.

Configuration and initial values (`tests/wide.txt`):

	# of rs	Cycles in EX	Cycles in Mem	# of FUs
Integer adder	8	1		4
FP adder	2	2		1
FP multiplier	1	4		1
Load/store unit	2	1	2	1
ROB entries = 16				

Instructions (the number is the instruction index that branch offsets count from):

```

0  Addi R1, R0, 1
1  Addi R2, R0, 2
2  Addi R3, R0, 3
3  Addi R4, R0, 4
4  Addi R5, R0, 5
5  Addi R6, R0, 6
6  Addi R7, R0, 7
7  Addi R8, R0, 8

```

Final non-zero registers and memory words (the same at every width): R1 = 1, R2 = 2, R3 = 3, R4 = 4, R5 = 5, R6 = 6, R7 = 7, R8 = 8.

Figure 13. `wide`, issue width 1 (CDB buses 1, commit width 1): 11 cycles, 8 committed instructions, IPC 0.73, 0 squashed.

	ISSUE	EX	MEM	WB	COMMIT	NOTE
Addi R1, R0, 1	[1]	[2, 2]	[]	[3]	[4]	
Addi R2, R0, 2	[2]	[3, 3]	[]	[4]	[5]	
Addi R3, R0, 3	[3]	[4, 4]	[]	[5]	[6]	
Addi R4, R0, 4	[4]	[5, 5]	[]	[6]	[7]	
Addi R5, R0, 5	[5]	[6, 6]	[]	[7]	[8]	
Addi R6, R0, 6	[6]	[7, 7]	[]	[8]	[9]	
Addi R7, R0, 7	[7]	[8, 8]	[]	[9]	[10]	
Addi R8, R0, 8	[8]	[9, 9]	[]	[10]	[11]	

Figure 14. `wide`, issue width 2 (CDB buses 2, commit width 2): 7 cycles, 8 committed instructions, IPC 1.14, 0 squashed.

	ISSUE	EX	MEM	WB	COMMIT	NOTE
Addi R1, R0, 1	[1]	[2, 2]	[]	[3]	[4]	
Addi R2, R0, 2	[1]	[2, 2]	[]	[3]	[4]	
Addi R3, R0, 3	[2]	[3, 3]	[]	[4]	[5]	
Addi R4, R0, 4	[2]	[3, 3]	[]	[4]	[5]	
Addi R5, R0, 5	[3]	[4, 4]	[]	[5]	[6]	
Addi R6, R0, 6	[3]	[4, 4]	[]	[5]	[6]	
Addi R7, R0, 7	[4]	[5, 5]	[]	[6]	[7]	
Addi R8, R0, 8	[4]	[5, 5]	[]	[6]	[7]	

Figure 15. `wide`, issue width 3 (CDB buses 3, commit width 3): 6 cycles, 8 committed instructions, IPC 1.33, 0 squashed.

	ISSUE	EX	MEM	WB	COMMIT	NOTE
Addi R1, R0, 1	[1]	[2, 2]	[]	[3]	[4]	
Addi R2, R0, 2	[1]	[2, 2]	[]	[3]	[4]	
Addi R3, R0, 3	[1]	[2, 2]	[]	[3]	[4]	
Addi R4, R0, 4	[2]	[3, 3]	[]	[4]	[5]	
Addi R5, R0, 5	[2]	[3, 3]	[]	[4]	[5]	
Addi R6, R0, 6	[2]	[3, 3]	[]	[4]	[5]	
Addi R7, R0, 7	[3]	[4, 4]	[]	[5]	[6]	
Addi R8, R0, 8	[3]	[4, 4]	[]	[5]	[6]	

Figure 16. *wide*, issue width 4 (CDB buses 4, commit width 4): 5 cycles, 8 committed instructions, IPC 1.60, 0 squashed.

	ISSUE	EX	MEM	WB	COMMIT	NOTE
Addi R1, R0, 1	[1]	[2, 2]	[]	[3]	[4]	
Addi R2, R0, 2	[1]	[2, 2]	[]	[3]	[4]	
Addi R3, R0, 3	[1]	[2, 2]	[]	[3]	[4]	
Addi R4, R0, 4	[1]	[2, 2]	[]	[3]	[4]	
Addi R5, R0, 5	[2]	[3, 3]	[]	[4]	[5]	
Addi R6, R0, 6	[2]	[3, 3]	[]	[4]	[5]	
Addi R7, R0, 7	[2]	[3, 3]	[]	[4]	[5]	
Addi R8, R0, 8	[2]	[3, 3]	[]	[4]	[5]	

3.7 Test case 5: *memory*

Loads and stores. The slow Mult.d keeps the stores from committing early, so loads can forward from them. The store to 0(R2) has no address until the Addi finishes; Ld F5, 0(R0) does not wait for it (as the handout says) and reads memory, Ld F4, 4(R1) has that store's address and forwards from it once the address is known. Stores write memory at commit.

Configuration and initial values (*tests/memory.txt*):

```

Integer adder    2    2    1
FP adder        2    2    1
FP multiplier    1    6    1
Load/store unit 6    1    3    1
ROB entries = 16

R1=8, F1=1.5, F2=2.5
Mem[0]=7.0, Mem[8]=10.0, Mem[12]=20.0

```

Instructions (the number is the instruction index that branch offsets count from):

```

0 Mult.d F6, F1, F2
1 Sd F1, 0(R1)
2 Ld F3, 0(R1)
3 Addi R2, R1, 4
4 Sd F2, 0(R2)
5 Ld F5, 0(R0)
6 Ld F4, 4(R1)

```

Final non-zero registers and memory words (the same at every width): R1 = 8, R2 = 12, F1 = 1.5, F2 = 2.5, F3 = 1.5, F4 = 2.5, F5 = 7.0, F6 = 3.75, Mem[0] = 7.0, Mem[8] = 1.5, Mem[12] = 2.5.

Figure 17. *memory*, issue width 1 (CDB buses 1, commit width 1): 16 cycles, 7 committed instructions, IPC 0.44, 0 squashed.

	ISSUE	EX	MEM	WB	COMMIT	NOTE
Mult.d F6, F1, F2	[1]	[2, 7]	[]	[8]	[9]	
Sd F1, 0(R1)	[2]	[3, 3]	[11, 13]	[]	[11]	
Ld F3, 0(R1)	[3]	[4, 4]	[5, 5]	[6]	[12]	forwarded
Addi R2, R1, 4	[4]	[5, 6]	[]	[7]	[13]	
Sd F2, 0(R2)	[5]	[8, 8]	[14, 16]	[]	[14]	
Ld F5, 0(R0)	[6]	[7, 7]	[8, 10]	[11]	[15]	
Ld F4, 4(R1)	[7]	[9, 9]	[10, 10]	[12]	[16]	forwarded

Figure 18. `memory`, issue width 2 (CDB buses 2, commit width 2): 14 cycles, 7 committed instructions, IPC 0.50, 0 squashed.

	ISSUE	EX	MEM	WB	COMMIT	NOTE
Mult.d F6, F1, F2	[1]	[2, 7]	[]	[8]	[9]	
Sd F1, 0(R1)	[1]	[2, 2]	[9, 11]	[]	[9]	
Ld F3, 0(R1)	[2]	[3, 3]	[4, 4]	[5]	[10]	forwarded
Addi R2, R1, 4	[2]	[3, 4]	[]	[5]	[10]	
Sd F2, 0(R2)	[3]	[6, 6]	[12, 14]	[]	[12]	
Ld F5, 0(R0)	[3]	[4, 4]	[5, 7]	[8]	[12]	
Ld F4, 4(R1)	[4]	[5, 5]	[7, 7]	[9]	[13]	forwarded

Figure 19. `memory`, issue width 3 (CDB buses 3, commit width 3): 14 cycles, 7 committed instructions, IPC 0.50, 0 squashed.

	ISSUE	EX	MEM	WB	COMMIT	NOTE
Mult.d F6, F1, F2	[1]	[2, 7]	[]	[8]	[9]	
Sd F1, 0(R1)	[1]	[2, 2]	[9, 11]	[]	[9]	
Ld F3, 0(R1)	[1]	[3, 3]	[4, 4]	[5]	[9]	forwarded
Addi R2, R1, 4	[2]	[3, 4]	[]	[5]	[10]	
Sd F2, 0(R2)	[2]	[6, 6]	[12, 14]	[]	[12]	
Ld F5, 0(R0)	[2]	[4, 4]	[5, 7]	[8]	[12]	
Ld F4, 4(R1)	[3]	[5, 5]	[7, 7]	[8]	[12]	forwarded

Figure 20. `memory`, issue width 4 (CDB buses 4, commit width 4): 14 cycles, 7 committed instructions, IPC 0.50, 0 squashed.

	ISSUE	EX	MEM	WB	COMMIT	NOTE
Mult.d F6, F1, F2	[1]	[2, 7]	[]	[8]	[9]	
Sd F1, 0(R1)	[1]	[2, 2]	[9, 11]	[]	[9]	
Ld F3, 0(R1)	[1]	[3, 3]	[4, 4]	[5]	[9]	forwarded
Addi R2, R1, 4	[1]	[2, 3]	[]	[4]	[9]	
Sd F2, 0(R2)	[2]	[5, 5]	[12, 14]	[]	[12]	
Ld F5, 0(R0)	[2]	[4, 4]	[5, 7]	[8]	[12]	
Ld F4, 4(R1)	[2]	[6, 6]	[7, 7]	[8]	[12]	forwarded

3.8 Test case 6: `memory_violation`

A load goes ahead of an older store whose address is not known yet, as the handout allows, but the store turns out to write the same address (8): the load read the old value 10.0. In the cycle after the store's address is calculated the conflict is found; the load and the Add.d that used its value are squashed and fetched again two cycles later. Final F2 = 1.5, F3 = 3.0.

Configuration and initial values (`tests/memory_violation.txt`):

```

# of rs    Cycles in EX    Cycles in Mem    # of FUs
Integer adder  2          4                      1
FP adder      2          2                      1
FP multiplier  1          4                      1
Load/store unit 4          1          2          1
ROB entries = 16

R1=8, F1=1.5
Mem[8]=10.0

```

Instructions (the number is the instruction index that branch offsets count from):

```

0  Addi R2, R1, 0
1  Sd F1, 0(R2)
2  Ld F2, 0(R1)
3  Add.d F3, F2, F2

```

Final non-zero registers and memory words (the same at every width): R1 = 8, R2 = 8, F1 = 1.5, F2 = 1.5, F3 = 3.0, Mem[8] = 1.5.

Figure 21. `memory_violation`, issue width 1 (CDB buses 1, commit width 1): 18 cycles, 4 committed instructions, IPC 0.22, 2 squashed.

	ISSUE	EX	MEM	WB	COMMIT	NOTE
Addi R2, R1, 0	[1]	[2, 5]	[]	[6]	[7]	
Sd F1, 0(R2)	[2]	[7, 7]	[8, 9]	[]	[8]	
Ld F2, 0(R1)	[10]	[11, 11]	[12, 13]	[14]	[15]	replayed
Add.d F3, F2, F2	[11]	[15, 16]	[]	[17]	[18]	

Figure 22. `memory_violation`, issue width 2 (CDB buses 2, commit width 2): 18 cycles, 4 committed instructions, IPC 0.22, 2 squashed.

	ISSUE	EX	MEM	WB	COMMIT	NOTE
Addi R2, R1, 0	[1]	[2, 5]	[]	[6]	[7]	
Sd F1, 0(R2)	[1]	[7, 7]	[8, 9]	[]	[8]	
Ld F2, 0(R1)	[10]	[11, 11]	[12, 13]	[14]	[15]	replayed
Add.d F3, F2, F2	[10]	[15, 16]	[]	[17]	[18]	

Figure 23. `memory_violation`, issue width 3 (CDB buses 3, commit width 3): 18 cycles, 4 committed instructions, IPC 0.22, 2 squashed.

	ISSUE	EX	MEM	WB	COMMIT	NOTE
Addi R2, R1, 0	[1]	[2, 5]	[]	[6]	[7]	
Sd F1, 0(R2)	[1]	[7, 7]	[8, 9]	[]	[8]	
Ld F2, 0(R1)	[10]	[11, 11]	[12, 13]	[14]	[15]	replayed
Add.d F3, F2, F2	[10]	[15, 16]	[]	[17]	[18]	

Figure 24. `memory_violation`, issue width 4 (CDB buses 4, commit width 4): 18 cycles, 4 committed instructions, IPC 0.22, 2 squashed.

	ISSUE	EX	MEM	WB	COMMIT	NOTE
Addi R2, R1, 0	[1]	[2, 5]	[]	[6]	[7]	
Sd F1, 0(R2)	[1]	[7, 7]	[8, 9]	[]	[8]	
Ld F2, 0(R1)	[10]	[11, 11]	[12, 13]	[14]	[15]	replayed
Add.d F3, F2, F2	[10]	[15, 16]	[]	[17]	[18]	

3.9 Test case 7: `loop`

A loop that runs twice. The empty BTB predicts the first Bne not taken and the 1-bit predictor then predicts the last one taken: both are mispredicted.

Configuration and initial values (`tests/loop.txt`):

	# of rs	Cycles in EX	Cycles in Mem	# of FUs
Integer adder	4	1		2
FP adder	2	2		1
FP multiplier	1	4		1
Load/store unit	2	1	2	1
ROB entries = 16				
R1=2				

Instructions (the number is the instruction index that branch offsets count from):

```
0  Addi R1, R1, -1
1  Bne R1, R0, -2
2  Addi R2, R2, 5
3  Add R3, R2, R2
```

Final non-zero registers and memory words (the same at every width): R2 = 5, R3 = 10.

Figure 25. `loop`, issue width 1 (CDB buses 1, commit width 1): 16 cycles, 6 committed instructions, IPC 0.38, 4 squashed.

	ISSUE	EX	MEM	WB	COMMIT	NOTE
Addi R1, R1, -1	[1]	[2, 2]	[]	[3]	[4]	
Bne R1, R0, -2	[2]	[4, 4]	[]	[]	[5]	taken, mispredicted
Addi R1, R1, -1	[6]	[7, 7]	[]	[8]	[9]	
Bne R1, R0, -2	[7]	[9, 9]	[]	[]	[10]	not taken, mispredicted
Addi R2, R2, 5	[11]	[12, 12]	[]	[13]	[14]	
Add R3, R2, R2	[12]	[14, 14]	[]	[15]	[16]	

Figure 26. `loop`, issue width 2 (CDB buses 2, commit width 2): 16 cycles, 6 committed instructions, IPC 0.38, 5 squashed.

	ISSUE	EX	MEM	WB	COMMIT	NOTE
Addi R1, R1, -1	[1]	[2, 2]	[]	[3]	[4]	
Bne R1, R0, -2	[1]	[4, 4]	[]	[]	[5]	taken, mispredicted
Addi R1, R1, -1	[6]	[7, 7]	[]	[8]	[9]	
Bne R1, R0, -2	[6]	[9, 9]	[]	[]	[10]	not taken, mispredicted
Addi R2, R2, 5	[11]	[12, 12]	[]	[13]	[14]	
Add R3, R2, R2	[11]	[14, 14]	[]	[15]	[16]	

Figure 27. `loop`, issue width 3 (CDB buses 3, commit width 3): 16 cycles, 6 committed instructions, IPC 0.38, 5 squashed.

	ISSUE	EX	MEM	WB	COMMIT	NOTE
Addi R1, R1, -1	[1]	[2, 2]	[]	[3]	[4]	
Bne R1, R0, -2	[1]	[4, 4]	[]	[]	[5]	taken, mispredicted
Addi R1, R1, -1	[6]	[7, 7]	[]	[8]	[9]	
Bne R1, R0, -2	[6]	[9, 9]	[]	[]	[10]	not taken, mispredicted
Addi R2, R2, 5	[11]	[12, 12]	[]	[13]	[14]	
Add R3, R2, R2	[11]	[14, 14]	[]	[15]	[16]	

Figure 28. `loop`, issue width 4 (CDB buses 4, commit width 4): 16 cycles, 6 committed instructions, IPC 0.38, 5 squashed.

	ISSUE	EX	MEM	WB	COMMIT	NOTE
Addi R1, R1, -1	[1]	[2, 2]	[]	[3]	[4]	
Bne R1, R0, -2	[1]	[4, 4]	[]	[]	[5]	taken, mispredicted
Addi R1, R1, -1	[6]	[7, 7]	[]	[8]	[9]	
Bne R1, R0, -2	[6]	[9, 9]	[]	[]	[10]	not taken, mispredicted
Addi R2, R2, 5	[11]	[12, 12]	[]	[13]	[14]	
Add R3, R2, R2	[11]	[14, 14]	[]	[15]	[16]	

3.10 Test case 8: `daxpy`

$Y[i] = a * X[i] + Y[i]$ for $i = 0..3$ (X at address 0, Y at 32, a in F0). The loop branch is mispredicted in the first iteration (empty BTB) and the last one (the 1-bit predictor says taken); iterations overlap in between.

Configuration and initial values ([tests/daxpy.txt](#)):

	# of rs	Cycles in EX	Cycles in Mem	# of FUs
Integer adder	2	1		1
FP adder	3	2		1
FP multiplier	2	4		1
Load/store unit	4	1	2	1

ROB entries = 32

R3=16, F0=2.0
 Mem[0]=1.0, Mem[4]=2.0, Mem[8]=3.0, Mem[12]=4.0, Mem[32]=10.0, Mem[36]=20.0, Mem[40]=30.0, Mem[44]=40.0

Instructions (the number is the instruction index that branch offsets count from):

```

0  Ld F2, 0(R1)
1  Mult.d F4, F2, F0
2  Ld F6, 32(R1)
3  Add.d F6, F4, F6
4  Sd F6, 32(R1)
5  Addi R1, R1, 4
6  Bne R1, R3, -7

```

Final non-zero registers and memory words (the same at every width): R1 = 16, R3 = 16, F0 = 2.0, F2 = 4.0, F4 = 8.0, F6 = 48.0, Mem[0] = 1.0, Mem[4] = 2.0, Mem[8] = 3.0, Mem[12] = 4.0, Mem[32] = 12.0, Mem[36] = 24.0, Mem[40] = 36.0, Mem[44] = 48.0.

Figure 29. [daxpy](#), issue width 1 (CDB buses 1, commit width 1): 41 cycles, 28 committed instructions, IPC 0.68, 2 squashed.

	ISSUE	EX	MEM	WB	COMMIT	NOTE
Ld F2, 0(R1)	[1]	[2, 2]	[3, 4]	[5]	[6]	
Mult.d F4, F2, F0	[2]	[6, 9]	[]	[10]	[11]	
Ld F6, 32(R1)	[3]	[4, 4]	[5, 6]	[7]	[12]	
Add.d F6, F4, F6	[4]	[11, 12]	[]	[13]	[14]	
Sd F6, 32(R1)	[5]	[6, 6]	[17, 18]	[]	[17]	
Addi R1, R1, 4	[6]	[7, 7]	[]	[8]	[18]	
Bne R1, R3, -7	[7]	[9, 9]	[]	[]	[19]	taken, mispredicted
Ld F2, 0(R1)	[11]	[12, 12]	[13, 14]	[15]	[20]	
Mult.d F4, F2, F0	[12]	[16, 19]	[]	[20]	[21]	
Ld F6, 32(R1)	[13]	[14, 14]	[15, 16]	[17]	[22]	
Add.d F6, F4, F6	[14]	[21, 22]	[]	[23]	[24]	
Sd F6, 32(R1)	[15]	[16, 16]	[25, 26]	[]	[25]	
Addi R1, R1, 4	[16]	[17, 17]	[]	[18]	[26]	
Bne R1, R3, -7	[17]	[19, 19]	[]	[]	[27]	taken
Ld F2, 0(R1)	[18]	[19, 19]	[20, 21]	[22]	[28]	
Mult.d F4, F2, F0	[19]	[23, 26]	[]	[27]	[29]	
Ld F6, 32(R1)	[20]	[21, 21]	[22, 23]	[24]	[30]	
Add.d F6, F4, F6	[21]	[28, 29]	[]	[30]	[31]	
Sd F6, 32(R1)	[22]	[23, 23]	[32, 33]	[]	[32]	
Addi R1, R1, 4	[23]	[24, 24]	[]	[25]	[33]	
Bne R1, R3, -7	[24]	[26, 26]	[]	[]	[34]	taken
Ld F2, 0(R1)	[25]	[26, 26]	[27, 28]	[29]	[35]	
Mult.d F4, F2, F0	[26]	[30, 33]	[]	[34]	[36]	
Ld F6, 32(R1)	[27]	[28, 28]	[29, 30]	[31]	[37]	
Add.d F6, F4, F6	[28]	[35, 36]	[]	[37]	[38]	
Sd F6, 32(R1)	[29]	[30, 30]	[39, 40]	[]	[39]	
Addi R1, R1, 4	[30]	[31, 31]	[]	[32]	[40]	
Bne R1, R3, -7	[31]	[33, 33]	[]	[]	[41]	not taken, mispredicted

Figure 30. *daxpy*, issue width 2 (CDB buses 2, commit width 2): 32 cycles, 28 committed instructions, IPC 0.88, 4 squashed.

	ISSUE	EX	MEM	WB	COMMIT	NOTE
Ld F2, 0(R1)	[1]	[2, 2]	[3, 4]	[5]	[6]	
Mult.d F4, F2, F0	[1]	[6, 9]	[]	[10]	[11]	
Ld F6, 32(R1)	[2]	[3, 3]	[5, 6]	[7]	[11]	
Add.d F6, F4, F6	[2]	[11, 12]	[]	[13]	[14]	
Sd F6, 32(R1)	[3]	[4, 4]	[18, 19]	[]	[18]	
Addi R1, R1, 4	[3]	[4, 4]	[]	[5]	[18]	
Bne R1, R3, -7	[4]	[6, 6]	[]	[]	[19]	taken, mispredicted
Ld F2, 0(R1)	[8]	[9, 9]	[10, 11]	[12]	[19]	
Mult.d F4, F2, F0	[8]	[13, 16]	[]	[17]	[20]	
Ld F6, 32(R1)	[9]	[10, 10]	[12, 13]	[14]	[20]	
Add.d F6, F4, F6	[9]	[18, 19]	[]	[20]	[21]	
Sd F6, 32(R1)	[10]	[11, 11]	[24, 25]	[]	[24]	
Addi R1, R1, 4	[10]	[11, 11]	[]	[12]	[24]	
Bne R1, R3, -7	[11]	[13, 13]	[]	[]	[25]	taken
Ld F2, 0(R1)	[12]	[13, 13]	[14, 15]	[16]	[25]	
Mult.d F4, F2, F0	[12]	[17, 20]	[]	[21]	[26]	
Ld F6, 32(R1)	[14]	[15, 15]	[16, 17]	[18]	[26]	
Add.d F6, F4, F6	[14]	[22, 23]	[]	[24]	[27]	
Sd F6, 32(R1)	[16]	[17, 17]	[27, 28]	[]	[27]	
Addi R1, R1, 4	[16]	[17, 17]	[]	[18]	[28]	
Bne R1, R3, -7	[17]	[19, 19]	[]	[]	[28]	taken
Ld F2, 0(R1)	[18]	[19, 19]	[20, 21]	[22]	[29]	
Mult.d F4, F2, F0	[18]	[23, 26]	[]	[27]	[29]	
Ld F6, 32(R1)	[19]	[20, 20]	[22, 23]	[24]	[30]	
Add.d F6, F4, F6	[19]	[28, 29]	[]	[30]	[31]	
Sd F6, 32(R1)	[22]	[23, 23]	[31, 32]	[]	[31]	
Addi R1, R1, 4	[22]	[23, 23]	[]	[25]	[32]	
Bne R1, R3, -7	[23]	[26, 26]	[]	[]	[32]	not taken, mispredicted

Figure 31. *daxpy*, issue width 3 (CDB buses 3, commit width 3): 31 cycles, 28 committed instructions, IPC 0.90, 4 squashed.

	ISSUE	EX	MEM	WB	COMMIT	NOTE
Ld F2, 0(R1)	[1]	[2, 2]	[3, 4]	[5]	[6]	
Mult.d F4, F2, F0	[1]	[6, 9]	[]	[10]	[11]	
Ld F6, 32(R1)	[1]	[3, 3]	[5, 6]	[7]	[11]	
Add.d F6, F4, F6	[2]	[11, 12]	[]	[13]	[14]	
Sd F6, 32(R1)	[2]	[4, 4]	[17, 18]	[]	[17]	
Addi R1, R1, 4	[2]	[3, 3]	[]	[4]	[17]	
Bne R1, R3, -7	[3]	[5, 5]	[]	[]	[17]	taken, mispredicted
Ld F2, 0(R1)	[7]	[8, 8]	[9, 10]	[11]	[18]	
Mult.d F4, F2, F0	[7]	[12, 15]	[]	[16]	[18]	
Ld F6, 32(R1)	[7]	[9, 9]	[11, 12]	[13]	[18]	
Add.d F6, F4, F6	[8]	[17, 18]	[]	[19]	[20]	
Sd F6, 32(R1)	[8]	[10, 10]	[23, 24]	[]	[23]	
Addi R1, R1, 4	[8]	[9, 9]	[]	[10]	[23]	
Bne R1, R3, -7	[9]	[11, 11]	[]	[]	[23]	taken
Ld F2, 0(R1)	[11]	[12, 12]	[13, 14]	[15]	[24]	
Mult.d F4, F2, F0	[11]	[16, 19]	[]	[20]	[24]	
Ld F6, 32(R1)	[13]	[14, 14]	[15, 16]	[17]	[24]	
Add.d F6, F4, F6	[13]	[21, 22]	[]	[23]	[25]	
Sd F6, 32(R1)	[15]	[16, 16]	[25, 26]	[]	[25]	
Addi R1, R1, 4	[15]	[16, 16]	[]	[17]	[25]	
Bne R1, R3, -7	[15]	[18, 18]	[]	[]	[26]	taken
Ld F2, 0(R1)	[17]	[18, 18]	[19, 20]	[21]	[26]	
Mult.d F4, F2, F0	[17]	[22, 25]	[]	[26]	[27]	
Ld F6, 32(R1)	[18]	[19, 19]	[21, 22]	[23]	[27]	
Add.d F6, F4, F6	[18]	[27, 28]	[]	[29]	[30]	
Sd F6, 32(R1)	[21]	[22, 22]	[30, 31]	[]	[30]	
Addi R1, R1, 4	[21]	[22, 22]	[]	[23]	[30]	
Bne R1, R3, -7	[21]	[24, 24]	[]	[]	[31]	not taken, mispredicted

Figure 32. `daxpy`, issue width 4 (CDB buses 4, commit width 4): 31 cycles, 28 committed instructions, IPC 0.90, 4 squashed.

	ISSUE	EX	MEM	WB	COMMIT	NOTE
Ld F2, 0(R1)	[1]	[2, 2]	[3, 4]	[5]	[6]	
Mult.d F4, F2, F0	[1]	[6, 9]	[]	[10]	[11]	
Ld F6, 32(R1)	[1]	[3, 3]	[5, 6]	[7]	[11]	
Add.d F6, F4, F6	[1]	[11, 12]	[]	[13]	[14]	
Sd F6, 32(R1)	[2]	[4, 4]	[17, 18]	[]	[17]	
Addi R1, R1, 4	[2]	[3, 3]	[]	[4]	[17]	
Bne R1, R3, -7	[2]	[5, 5]	[]	[]	[17]	taken, mispredicted
Ld F2, 0(R1)	[7]	[8, 8]	[9, 10]	[11]	[17]	
Mult.d F4, F2, F0	[7]	[12, 15]	[]	[16]	[18]	
Ld F6, 32(R1)	[7]	[9, 9]	[11, 12]	[13]	[18]	
Add.d F6, F4, F6	[7]	[17, 18]	[]	[19]	[20]	
Sd F6, 32(R1)	[8]	[10, 10]	[23, 24]	[]	[23]	
Addi R1, R1, 4	[8]	[9, 9]	[]	[10]	[23]	
Bne R1, R3, -7	[8]	[11, 11]	[]	[]	[23]	taken
Ld F2, 0(R1)	[11]	[12, 12]	[13, 14]	[15]	[23]	
Mult.d F4, F2, F0	[11]	[16, 19]	[]	[20]	[24]	
Ld F6, 32(R1)	[13]	[14, 14]	[15, 16]	[17]	[24]	
Add.d F6, F4, F6	[13]	[21, 22]	[]	[23]	[24]	
Sd F6, 32(R1)	[15]	[16, 16]	[25, 26]	[]	[25]	
Addi R1, R1, 4	[15]	[16, 16]	[]	[17]	[25]	
Bne R1, R3, -7	[15]	[18, 18]	[]	[]	[25]	taken
Ld F2, 0(R1)	[17]	[18, 18]	[19, 20]	[21]	[25]	
Mult.d F4, F2, F0	[17]	[22, 25]	[]	[26]	[27]	
Ld F6, 32(R1)	[18]	[19, 19]	[21, 22]	[23]	[27]	
Add.d F6, F4, F6	[18]	[27, 28]	[]	[29]	[30]	
Sd F6, 32(R1)	[21]	[22, 22]	[30, 31]	[]	[30]	
Addi R1, R1, 4	[21]	[22, 22]	[]	[23]	[30]	
Bne R1, R3, -7	[21]	[24, 24]	[]	[]	[30]	not taken, mispredicted

3.11 What limits the gain from a wider issue

- **Dependences.** In `chain` every instruction needs the previous result: 22 cycles at every width and in every setting.
- **The CDB.** In `wide`, 4 integer adders finish up to 4 instructions in the same cycle, but with one CDB only one result per cycle is written back: 11 cycles at every width (`cdb_limit` is the same program with `CDB buses = 1` in the file).
- **Commit.** For `daxpy` at width 4 with one CDB, one commit per cycle gives 41 cycles; a commit width of 4 gives 33, with 4 instructions committing in the same cycle in cycles 17, 25, 27 and 32. With one CDB and one commit per cycle, `daxpy` takes 41 cycles at every width.
- **Memory.** One port and memory accesses of several cycles: in `daxpy` 12 accesses of 2 cycles each (Figures 29–32); in `sample_tc1` and `sample_tc2` each access takes 6 or 5 cycles.
- **Branches.** Each misprediction delays the correct instructions until 2 cycles after the branch's EX, and a branch predicted taken ends the issue group, so at most one loop iteration starts per cycle. In `loop` the 2 mispredictions decide the timing: 16 cycles at every width; a wider issue only fetches more wrong-path instructions (4 squashed at width 1, 5 at width 4).
- **Full structures.** In `structural` (1 reservation station per FU type, a 2-entry load/store queue, a 4-entry ROB) issue keeps stopping at the first instruction without a free entry: 20 → 18 cycles.

Appendix A. Problems in the original code

We copied the original code, added only a few lines at the end of `main.py` to print the final registers and memory, and ran it on small programs. Its configuration is fixed in `main.py` (FP multiply 15 cycles, FP add 4, memory 5; R1 = 12, F20 = 3.0). Each program below is in `tests/bugN_*.txt` with that configuration, and `compare_original.py` prints the original's table next to ours (Appendix D).

#	Program	Original code	Correct	Cause in the original code
B1	<pre>Mult.d F6 F20 F20, Mult.d F2 F20 F20, Add.d F2 F20 F20, three Mult.d that need F6, Add.d F4 F2 F2</pre>	F4 = 18.0	F4 = 12.0	<code>wb.py</code> wrote every result into the RAT. The slow <code>Mult.d F2</code> finished after the newer <code>Add.d F2</code> and replaced its RAT entry with 9.0; the last <code>Add.d</code> issued later and read 9.0
B2	<pre>Mult.d F2 F20 F20, Sd F20 0(R1), Add.d F4 F2 F2</pre>	the store commits and writes memory in cycles 4–8; the older <code>Mult.d</code> commits in cycle 18	the store commits after the <code>Mult.d</code>	<code>mem.py</code> sent a store to memory as soon as it was first in the load/store queue with its address and data. The check meant to stop this, <code>check_if_sd_committable</code> , was used without being called, so it was always true

#	Program	Original code	Correct	Cause in the original code
B3	<code>Ld F8 0(R2), Sd F20 4(R1), Ld F2 4(R1)</code>	the second load computes its address and gets the forwarded value in the same cycle (EX [4, 4], MEM [4]); the run stops in cycle 9 while the store writes memory until cycle 12, so address 16 is never written	forwarding one cycle after the address; address 16 holds 3.0 at the end	<code>mem</code> ran after <code>exe</code> in the same cycle and saw the new address; the main loop stopped when the ROB was empty
B4	<code>Mult.d F2 F20 F20, Ld F4 0(R1), Mult.d F6 F4 F20</code>	the first <code>Mult.d</code> takes 14 cycles (EX [2, 15])	15 cycles	when a waiting instruction entered a pipelined unit, <code>exe.py</code> called <code>fu_exe</code> a second time in the same cycle, which moved every instruction in that unit forward again (the sample report found this too)
B5	<code>Addi R1 R1 -4, Bne R1 R0 -8 (offset in bytes; -2 in our format), Add.d F4 F20 F20</code>	<code>IndexError</code> in <code>commit.py</code>	runs; R1 = 0, F4 = 6.0	<code>commit</code> read <code>ROB[0]</code> after the branch had emptied the ROB

Reading the code (not shown by a run) we also found: a reservation station chose the instruction with the lowest station number, not the oldest; the integer adder was modelled as pipelined; R0 could be renamed and written; the "# of FUs" column was ignored; and every data structure was a new `namedtuple` class per entry, with values stored as class attributes, so an unset field read back as a descriptor object instead of `None`.

Appendix B. All timing rules and where they come from

Handout: stated in the assignment PDF. **Original:** what the original code does, kept so that width 1 stays close to it. **Ours:** not specified (or the original was wrong); our choice. Appendix C.2 checks the limits and orderings of these rules on every committed instruction; the policies (which instruction goes first) are checked by the hand-computed tables (C.3).

B.1 Pipeline timing

Rule	Source
EX starts at least 1 cycle after ISSUE	Original
EX starts at least 1 cycle after every source operand's WB	Original
EX lasts the configured number of cycles: [s, s+L-1]	Handout
WB at least 1 cycle after the end of EX (MEM for loads)	Original
A load's MEM starts at least 1 cycle after its EX (address)	Original (the original broke it, B3)
COMMIT at least 1 cycle after WB (after EX for branches)	Original
A reservation station, ROB entry, queue entry or FU freed in cycle <i>c</i> can be used again in cycle <i>c</i> +1	Ours (the original reused ROB entries in the same cycle)
A reservation station is freed when its instruction starts EX	Original

B.2 Multiple issue

Rule	Source
Up to <i>W</i> instructions issue per cycle, in program order; issue stops at the first one without a free ROB entry, reservation station or queue entry	Handout ("1-4 instructions, in order")
A branch predicted taken is the last instruction issued in its cycle; the target is fetched in the next cycle	Ours
CDB buses: default <i>W</i> , or <code>CDB buses = N / --cdb N</code>	Ours, following the sample report (section 1.4); the handout lists one CDB
Commit width: default <i>W</i> , or <code>Commit width = N / --commit-width N</code>	Ours, following the sample report
"# of FUs" FUs per type; each can start one instruction per cycle	Handout (the original ignored the column)
FP adder and multiplier are pipelined; the integer adder is not; the address adders (the "# of FUs" of the load/store row) are not	Handout for the first three; Ours for the address adders (no difference when address calculation takes 1 cycle, as in every handout example)
When several instructions are ready for the same FU type, the oldest goes first	Ours (the original took the lowest station number)
When more results are waiting than there are CDB buses, the one that finished first goes first; ties go to the oldest instruction	Ours (the original used the order in which the stages added results)

B.3 Branches

Rule	Source
Beq / Bne use an integer adder reservation station and FU; offset in instructions: target = index + 1 + offset	Handout
8-entry BTB, index = word address of the branch mod 8 (instruction <i>i</i> is at byte address 4 <i>i</i>)	Handout
Each entry holds the PC of its branch, which is checked on lookup; a lookup with another PC predicts not taken	Ours: the handout does not say. Without it, branches 8 instructions apart use each other's target (tests/btb_alias.txt)
1-bit predictor per entry: predict what the branch did last time; empty entry: not taken	Handout (1-bit); Ours (initial value)
Predict at ISSUE; resolve and update the BTB at the end of EX	Handout
Misprediction found in cycle <i>n</i> : no issue in <i>n</i> +1, correct fetch in <i>n</i> +2	Handout
The wrong-path instructions are removed at the end of cycle <i>n</i> , so older instructions can use what they held from <i>n</i> +1	Ours (the handout counts the recovery as cycle <i>n</i> +1; this only matters for resources the wrong path held)
If two branches resolve wrongly in the same cycle, the older one squashes the younger	Ours

B.4 Loads, stores and memory

Rule	Source
Memory is 256 bytes; Ld and Sd move 4-byte single-precision values; addresses are byte addresses	Handout
A value at address <i>a</i> uses bytes <i>a</i> to <i>a</i> +3, little-endian; <i>a</i> can be any of 0 to 252, also not a multiple of 4; another address stops the run when the instruction commits	Ours (the handout's sample loads from address 18)
Loads and stores enter the queue in program order at ISSUE; a full queue stops issue	Handout
Address calculation on a dedicated adder, oldest ready entry first	Handout / Ours
A store commits only when every older instruction has committed (possibly earlier in the same cycle), its address and data are known, and the memory is free; it writes memory from that cycle for "Cycles in Mem" cycles, and its COMMIT cycle is the first of them	Handout (stores write at commit); Original (COMMIT = start of the write)
From the cycle after its address is known, a load looks at the older stores in the queue whose address is known; stores with no address yet do not stop it	Handout
If the youngest of those stores that overlaps the load is at the same address and has its data: forward it (rounded to single precision), MEM takes 1 cycle; without its data: wait	Handout
If that store overlaps the load only partly: wait until it has written memory	Ours (only possible with addresses that are not multiples of 4)
If no such store: read memory when it is free (one access at a time, not pipelined); the oldest such load goes first	Handout
A load leaves the queue in the cycle it gets its data; a store when it commits	Handout
Memory-order check: in the cycle after a store's address is calculated, every younger load that already got (or is getting) its data and overlaps the store is wrong, unless it forwarded from a younger store at the same address. The oldest such load and everything after it are squashed; the load is fetched again two cycles later, as after a branch	Ours (the handout gives no check; without it such a load keeps an old value)
A load and a committing store that want the memory in the same cycle: the load gets it	Ours

Appendix C. Verification

The original code came with no tests. The scripts in [verify/](#) check the simulator from several sides; [run_all.sh](#) runs all of them and exits with status 1 if any fails.

C.1 Functional reference

[reference.py](#) runs the program one instruction at a time, with no timing, renaming or speculation, on its own 256-byte memory (written again, not imported from the simulator). For every test, width and setting, [check_all.py](#) requires the same final integer registers, FP registers and memory bytes, and the same sequence of committed instructions. It shares only the input parser with the simulator.

A run that stops at the cycle limit passes only if the reference cannot finish the program either; then only the committed instructions so far can be compared, and the run is counted as "prefix", not "ok". Only [pdf_sample](#) is like that.

C.2 Timing rules

`check_all.py` also checks every committed instruction against the limits and orderings of the rules of Appendix B, written again from the tables rather than taken from the simulator's code; which instruction goes first when several could is left to the hand-computed tables (C.3). It reads the limits from the input file, and which units are pipelined from its own table. The rules checked:

- ISSUE and COMMIT in program order; at most W issues, the commit width of commits and the number of CDB buses of broadcasts per cycle;
- EX starts after ISSUE and after the WB of each source operand's producer, and has the configured length; WB after EX/MEM; COMMIT after WB (branches: after EX);
- loads: MEM after EX, 1 cycle if forwarded, else the memory latency; stores: MEM is $[\text{COMMIT}, \text{COMMIT} + \text{latency} - 1]$, COMMIT after the data's producer wrote back;
- no two memory accesses overlap in time; unpipelined FUs never hold more instructions than there are units; pipelined FUs start at most one instruction per unit per cycle;
- the ROB, each reservation station type and the load/store queue never hold more entries than configured;
- after a mispredicted branch, the next instruction issues at least 2 cycles after the branch's EX; after a branch predicted taken, the next instruction issues in a later cycle;
- a forwarded load forwarded from the youngest older store that overlaps it, which is at the same address, had its address and data, and had not committed yet; a load that read memory started after every older overlapping store finished writing.

It also checks that the printed registers and non-zero memory read back as the values the simulator holds. Only committed instructions appear in the table, so these checks cannot see squashed ones.

C.3 Hand-computed tables

C.1 and C.2 check that the output is consistent with the rules. They cannot tell that a rule was implemented differently from what we meant, for example "lowest station number" instead of "oldest". So for 7 small tests we computed the whole instruction table by hand at widths 1, 2 and 4 before running the simulator on them: 19 tables in `tests/golden/` (`squash_cdb` only at width 4). `check_all.py` compares the printed tables with them.

When the memory rule changed to "loads do not wait for stores with unknown addresses", we derived the three `memory` tables again by hand under the new rule, and the three `memory_violation` tables for the new check, before running the changed simulator; all six matched. One of the first 16 tables did not match on its first run (`dispatch_order`, width 4): our derivation said `Add R5, R1, R0` issues in cycle 1, but we had copied the width-2 row into the file. The simulator agreed with the derivation.

Example, test case 7 (`loop`) at width 1 (Figure 25):

- The first `Bne` needs R1, written back in cycle 3, so its EX is cycle 4. The BTB is empty, so it was predicted not taken, and the two instructions after it were fetched in cycles 3 and 4. In cycle 4 it turns out taken: those two are squashed, cycle 5 is the recovery cycle, and the loop body is fetched again in cycle 6 ($n + 2$).
- The second `Bne` finds its BTB entry (index 1, PC 1) saying "taken last time" and is predicted taken, so `Addi R1` is fetched again in cycle 8. In cycle 9 the branch turns out not taken ($R1 = 0$): squash, nothing in cycle 10, `Addi R2` in cycle 11.
- `Add R3` needs R2, written back in cycle 13, so its EX is cycle 14.

C.4 Random programs

`fuzz.py` generates programs of 3 to 35 instructions with up to two counted loops, forward branches, loads and stores to a few overlapping addresses (some not multiples of 4), stores followed by loads of the same address through another base register whose value is computed late (so loads run ahead of stores and some are replayed), and values that single precision has to round (0.1, 3.4, 16777217, 1e-7). Each gets a random configuration (1–4 stations and 1–2 units per FU type, latencies 1–6, ROB of 2–12 entries, and half the time a CDB count or commit width of 1–4 that can be below the issue width). Programs the reference cannot finish in 2000 instructions, or that access an invalid address, are skipped. Each program runs at widths 1–4 through the checks of C.1 and C.2. In `results/fuzz.txt`: 2000 programs, 8000 runs, 0 failures.

C.5 Mutation test

A check is only useful if it fails when the code is wrong. `mutants.py` copies the code, changes one line, and runs `check_all.py` and `fuzz.py` (300 programs) on the copy. All 25 mutants are caught (`results/mutants.txt`):

Mistake put into the code	Caught by tests	Caught by fuzz
RAT not recovered after a misprediction	yes	yes
correct fetch at $n+1$ instead of $n+2$	yes	yes
reservation stations of wrong-path instructions not cleared	yes	yes
wrong-path results left waiting for the CDB	yes	no
RAT written by every broadcast (bug B1)	yes	yes
no limit on CDB buses	yes	yes
commit in the same cycle as WB	yes	yes
store commits while the memory is busy	yes	yes
no limit on commit width	yes	yes
forward from the oldest overlapping store instead of the youngest	no	yes
loads wait for every older store address (the rule before the change)	yes	no
partly overlapping stores ignored (only the same address counts)	yes	yes
no memory-order check	yes	yes
replay starts after the load instead of at it	yes	yes
a load that forwarded is never replayed	yes	no
forwarded value not rounded to single precision	yes	yes
register values printed with 6 decimals ($1e-07$ printed as 0.0)	yes	yes
forwarding in the same cycle as the address calculation (bug B3)	yes	yes
EX in the same cycle as ISSUE	yes	yes
unpipelined integer adder treated as pipelined	no	yes
BTB ignores the PC stored in the entry	yes	no
a branch predicted taken does not end the issue group	yes	yes
issue does not stop at a full ROB	yes	yes
youngest ready instruction dispatched first	yes	no
B_{eq} resolved like B_{ne}	yes	yes

The mutants that only change timing, not results (BTB tags, dispatch order, waiting for store addresses), are caught by the hand-computed tables. Some checks were too weak at first and were strengthened: the checker used to read which units are pipelined from the simulator itself; a run stopped by the cycle limit used to count as passed even when the program does end (now only "prefix" for a program that never ends, and a failure otherwise); `run_all.sh` used to exit with status 0 even when a check failed; and values were printed with six decimals, so $1e-07$ showed as 0.0 and no check looked at the printed values (now C.2 does, and `tests/precision.txt` has the case).

Appendix D. Width 1 compared with the original code

`compare_original.py` runs the original code (taken from the repository history) and ours at width 1 on the five inputs the original can run (`results/compare_original.txt`). Every difference has one of these causes:

Cause	Where it shows
CDB ties: a load finishing MEM and an $Addi$ finishing EX in cycle 7 both want the CDB in cycle 8. The original takes the $Addi$ (its stage order added it first), we take the older load	<code>original.txt</code> : $Ld\ F2\ WB\ 8$ instead of 9, and everything that depends on it one cycle earlier
Branch prediction instead of stopping fetch	<code>original.txt</code> : the original issues after each B_{ne} one cycle after it resolves. We lose 2 cycles on the first and last B_{ne} (mispredicted) and nothing on the middle one (predicted taken correctly)
Branches take a commit slot	<code>original.txt</code> : the original removes a B_{ne} without a commit cycle; ours commits it like any instruction. The three B_{ne} add three cycles at the end and the CDB tie saves one: 41 cycles instead of 39

Cause	Where it shows
B1, B4: pipelined units advanced twice	bug1, bug4: the original's <code>Mult.d</code> take 14 cycles, two start EX in the same cycle in one unit
B2, B3: stores and forwarding	bug2, bug3: stores commit after older instructions, forwarding a cycle after EX

Appendix E. Comparison with the sample report

We ran the sample report's two test cases unchanged (test cases 2 and 3 in section 3).

Test case 1 of the sample report at width 1 ends in cycle 36 in both. Rows differ where two results want the CDB in the same cycle: the sample report's tie order puts the second `Ld F1` before `Mult.d F2 F1 F1` (WB 16 and 17); we take the older instruction first (17 and 16), which moves the dependent `Add.d F4 F2 F4` one cycle earlier. At width 4 both end in cycle 33. There we differ in two places. The sample report reuses a reservation station in the cycle it is freed (the fifth `Mult.d` issues in cycle 10, ours in 11). And its second `Addi R1 R1 8` starts EX in cycle 4, the cycle in which the first `Addi R1 R1 8`, whose result it needs, writes back. In the other rows we checked, an instruction starts EX one cycle after its operand's WB (at width 1, the `Ld` after the first `Addi R1 R1 8`: WB 8, EX 9). We think cycle 4 is an error in the sample report; ours starts in cycle 5.

Test case 2 of the sample report: we take 49 cycles at width 1, the sample report 47. It has "# of FUs = 2" for the load/store unit, and the sample report lets two loads use memory at once (MEM [3, 7] and [7, 11]). The handout says memory is single-ported, so we read that column as the number of address adders, and the second load waits until cycle 8. Its `Addi R1 R1 8` also starts EX in cycle 9, the cycle its operand writes back.

The sample report says branches do not work at widths above 1. In ours they work at every width.